

4IM05 - Kaggle Challenge Report

Agshay Nadanakumar
Télécom Paris

Avril 2026

Résumé

Ce rapport présente ma solution au challenge de classification automatique de globules blancs (WBC) à partir d'images microscopiques. Le dataset contient 28 901 images réparties en 13 classes de cellules sanguines. J'ai adopté une démarche progressive, qui part d'une pipeline de Machine Learning classique avec extraction manuelle de features vers une pipeline de Deep Learning fondée sur le transfer learning.

1 Introduction

Les globules blancs (ou leucocytes) constituent un pilier fondamental du système immunitaire humain. Ils assurent la défense de l'organisme contre les agents pathogènes (bactéries, virus et parasites) et jouent un rôle central dans la détection de nombreuses pathologies hématologiques, dont la leucémie [1].

L'identification et la classification de celles-ci à partir de frottis sanguins microscopiques est une étape diagnostique importante en hématologie clinique. Traditionnellement réalisée manuellement par des spécialistes, cette tâche pourrait être liée à des erreurs dépendant de la subjectivité de l'observateur, de la fatigue et aux variations de coloration entre échantillons [1]. L'automatisation de ce processus par des techniques de traitement d'images et de machine learning représente donc un enjeu majeur pour améliorer la rapidité, la reproductibilité et la fiabilité du diagnostic.

Dans le cadre du cours 4IM05 à Télécom Paris, j'ai participé à un challenge de classification de WBC présentant deux difficultés majeures :

- Un nombre de classes élevé (13 vs 4/5 classes habituellement selon [3, 4])
- Un déséquilibre extrême entre les classes (SNE : 13 015 images vs PLY : 11 images)

Afin d'évaluer chaque classe indépendamment de sa taille, on va utiliser le F1-score macro qui

mesure la performance sur les différentes classes, peu importe le nombre d'éléments dans la classe.

$$F1_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C F1_c$$

$$F1_c = 2 \cdot \frac{\text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}$$

Ma démarche s'articule en deux phases. On va commencer par une pipeline de Machine Learning classique fondée sur l'extraction manuelle de features (géométriques, texturales, couleur et in-variantes) couplée à des classifieurs traditionnels (SVM, Random Forest, LDA). Face aux limites de cette approche sur 13 classes, on a développé ensuite une pipeline de Deep Learning qui exploite le transfer learning depuis ImageNet. Ce rapport décrit donc l'ensemble de cette démarche itérative.

2 État de l'art

La classification automatique des WBC à partir d'images microscopiques fait l'objet d'une littérature abondante depuis les années 2000. Al-Dulaimi et al. [1] proposent une revue exhaustive identifiant six étapes clés : acquisition, prétraitement, segmentation, extraction de features, classification et évaluation. On va structurer cet état de l'art selon la distinction entre approches à features manuelles et approches end-to-end par Deep Learning.

2.1 Features manuelles et Machine Learning classique

Les méthodes classiques suivent une pipeline en deux temps : extraction de descripteurs à partir du noyau segmenté, puis classification par un algorithme de ML. Les principales contributions sont :

- **Ghosh et al. [3]** : Utilisation de features géométriques (aire, périmètre, excentricité, circularité) extraites après segmentation par watershed, couplées à un classifieur Naïve Bayes. Accuracy de 83.2% sur 5 classes.

- **Rezatofghi et al. [4]** : Introduction des *Local Binary Patterns* (LBP) pour capturer les textures des noyaux, combinés avec des matrices de co-occurrence et un classifieur SVM. Accuracy de 86.10% sur 5 classes.
- **Su et al. (2014)** : Proposition du *Local Directional Pattern* (LDP), une variante directionnelle du LBP qui utilise 8 masques de Kirsch, testée avec SVM et MLP sur la base CellaVision (77–97%).
- **Habibzadeh et al. (2013)** : Utilisation de la *Dual-Tree Complex Wavelet Transform* (DTCWT) avec SVM et K-PCA pour la classification en 5 classes (76–84%).
- **Al-Dulaimi et al. (2018) [1]** : Proposition de features bispectrales invariantes (96.13%) et de L-moments via la projection de Radon (97.23%) pour une classification en 10 classes.
- **Rustam et al. [2]** : Combinaison de features RGB et de texture, sélectionnées par Chi2, avec un Random Forest et un oversampling SMOTE. Accuracy de 0.97 et F1 de 0.92 sur 5 classes, surpassant les modèles DL testés dans la même étude.

2.2 Deep Learning et transfer learning

Le Deep Learning offre une alternative où le réseau apprend ses propres représentations directement depuis les pixels, sans segmentation ni choix de features. Les principaux travaux sont :

- **Zhao et al. (2017)** : Combinaison de PRI-CoLBP avec CNN et Random Forest pour détecter basophiles et éosinophiles, puis CNN pour les autres types. Accuracy de 92.6% sur 5 classes.
- **Habibzadeh et al. (2018) [5]** : Utilisation de ResNet50 et Inception pré-entraînés sur ImageNet pour 4 classes. Accuracy de 99.84% (ResNet) et 99.46% (Inception), mais avec un coût computationnel très élevé (3 000 époques).
- **Sharma et al. (2022)** : DenseNet121 optimisé avec normalisation et data augmentation. Accuracy de 98.84% sur le dataset KBC.
- **Cheuque et al. (2022)** : Architecture multi-niveaux avec Faster R-CNN pour la détection de la région d'intérêt, puis MobileNet pour la classification. Accuracy de 98.4%.

Cependant, Al-Dulaimi et al. [1] soulignent que le DL nécessite de grandes quantités de données et présente un coût computationnel élevé. Rustam et

al. [2] confirment que sur un dataset déséquilibré, VGG16 et ResNet50 sous-performent par rapport au Random Forest sans oversampling. Ces observations vont guider notre démarche : commencer par le ML classique, puis exploiter le transfer learning DL avec une attention particulière à la gestion du déséquilibre.

3 Dataset

3.1 Description générale

Le dataset du challenge comprend 28 901 images microscopiques de globules blancs, déjà réparties en un ensemble d'entraînement/validation (70%, 20 231 images avec labels) et un ensemble de test (30%, 9 634 images sans labels).

Les 13 classes correspondent à différents types et stades de maturation des leucocytes : neutrophiles segmentés (SNE), lymphocytes (LY), monocytes (MO), éosinophiles (EO), basophiles (BA), lymphocytes variants (VLY), neutrophiles en bande (BNE), métamyélocytes (MMY), myélocytes (MY), promyélocytes (PMY), blastes (BL), plasmocytes (PC) et prolymphocytes (PLY).

3.2 Déséquilibre des classes

Le déséquilibre du dataset est l'un des défis centraux de ce challenge. La figure 1 présente la distribution complète. La classe SNE représente à elle seule 45% du dataset d'entraînement, tandis que PLY est uniquement constitué de 11 images (0.04%).

Classe	Nb images	%	Ratio vs PLY
SNE	13 015	45.0%	1 183×
LY	8 101	28.0%	736×
MO	2 746	9.5%	250×
BL	2 012	7.0%	183×
EO	861	3.0%	78×
MY	441	1.5%	40×
BA	415	1.4%	38×
BNE	391	1.4%	36×
VLY	366	1.3%	33×
MMY	360	1.2%	33×
PMY	114	0.4%	10×
PC	68	0.2%	6×
PLY	11	0.04%	1×
Total	28 901	100%	-

FIGURE 1 – Distribution des 13 classes dans l'ensemble d'entraînement.

Ce déséquilibre a des conséquences directes sur le choix de la métrique d'évaluation. L'accuracy globale est dominée par les classes majoritaires : un modèle qui prédit systématiquement SNE obtiendrait 45% d'accuracy. Le F1-score macro, en revanche, accorde un poids égal à chaque classe, ce qui rend la performance sur les classes rares (PLY, PC, PMY) tout aussi déterminante que sur SNE ou LY.

3.3 Conséquences pour la modélisation

Ce déséquilibre extrême impose plusieurs stratégies :

- **Oversampling** des classes minoritaires (duplication d'images + augmentation)
- **Pondération des classes** dans la fonction de perte (class weights)
- **WeightedRandomSampler** pour équilibrer les mini-batches
- Évaluation systématique par le **F1-score macro** plutôt que l'accuracy

4 Méthodologie : Pipeline ML classique

On commence par suivre la démarche classique décrite par Al-Dulaimi et al. [1] : segmentation des cellules, extraction manuelle de features, puis classification par des algorithmes de ML traditionnels.

4.1 Segmentation des images

On a implémenté une pipeline de segmentation en quatre étapes, illustrée par la Figure 2 :

1. **Image originale** : l'image microscopique brute montre la cellule d'intérêt entourée de globules rouges et du fond plasmatique.
2. **Seuillage HSV + Otsu** : conversion dans l'espace couleur HSV, puis seuillage d'Otsu sur le canal de saturation et le canal bleu pour isoler les régions fortement colorées (noyaux violets/bleus caractéristiques de la coloration de Wright).
3. **Opérations morphologiques** : ouverture (suppression du bruit) puis fermeture (comblement des trous) avec un élément structurant elliptique, afin d'obtenir un masque propre du noyau.
4. **Watershed** : raffinement final par l'algorithme de watershed pour séparer d'éventuelles cellules adjacentes et délimiter précisément les contours du noyau.

Le masque binaire résultant isole le noyau cellulaire, sur lequel l'ensemble des features sont ensuite extraites.

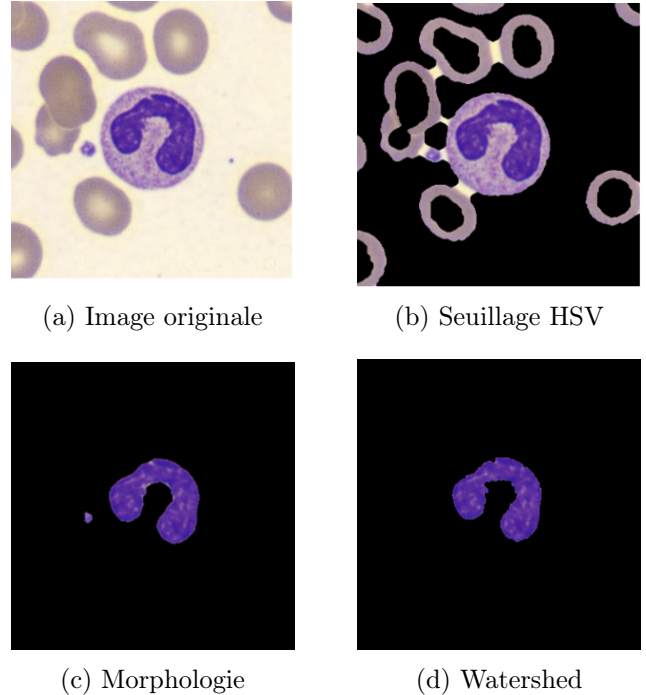


FIGURE 2 – Pipeline de segmentation du noyau WBC. (a) Image microscopique brute. (b) Seuillage d'Otsu dans l'espace HSV isolant les composantes violettes. (c) Nettoyage par opérations morphologiques (ouverture/fermeture). (d) Raffinement final par watershed.

4.2 Extraction de features

Nous avons extrait quatre familles de features, inspirées de la littérature :

Features géométriques (inspirées de Ghosh et al. [3]) : aire, périmètre, rayon équivalent, circularité ($4\pi A/P^2$), excentricité, compacité, solidité, étendue et rapport d'aspect. Ces descripteurs vont capturer la forme globale du noyau, qui varie énormément entre les types cellulaires (noyau bilobé des neutrophiles vs. noyau rond des lymphocytes).

Features de texture (LBP, LDP, PRI-CoLBP) : le *Local Binary Pattern* (LBP) uniforme [4] va encoder la texture locale en comparant chaque pixel à ses voisins. Le *Local Directional Pattern* (LDP) utilise 8 masques de Kirsch pour capturer les gradients directionnels. Le *PRICoLBP* capture les co-occurrences entre paires de LBP à des positions diamétralement opposées, qui va donc offrir une invariance en rotation [1].

Features couleur/intensité : statistiques (moyenne, écart-type, skewness, kurtosis) sur les canaux RGB et HSV de la région segmentée, complétées par des ratios de couleur normalisés. La couleur est particulièrement discriminante car les différents types de granules (éosinophiles, basophiles) présentent des affinités tinctoriales distinctes.

Features invariantes/statistiques : DT-CWT (approximée par des filtres de Gabor multi-échelle multi-orientation), features bispectrales (profil radial de la FFT 2D) et L-moments (L-mean, L-scale, L-skewness, L-kurtosis) via la projection de Radon, inspirés des travaux d'Al-Dulaimi et al. [1] qui rapportent jusqu'à 97.23% d'accuracy sur 10 classes.

4.3 Classifieurs ML

On va évaluer six classifieurs après standardisation des features, couvrant différentes familles d'algorithmes :

- **SVM (noyau RBF)** : classifieur le plus utilisé dans la littérature WBC selon Al-Dulaimi et al. [1], qui performe en grande dimension.
- **Random Forest** : ensemble d'arbres de décision, robuste au bruit et capable de capturer des interactions non-linéaires entre features.
- **LDA** : réduction de dimensionnalité supervisée couplée à la classification, utilisée par Al-Dulaimi et al. avec les L-moments [1].
- **K-PCA + SVM** : réduction non-linéaire par Kernel PCA suivie d'un SVM, inspirée de Habibzadeh et al. (2013).
- **Régression Logistique** : modèle linéaire multinomial, servant de baseline.
- **Arbre de Décision** : modèle interprétable, servant également de baseline.

Le meilleur F1 macro obtenu est de **0.459** (SVM), pour un score Kaggle de **0.4320**. L'analyse détaillée des performances par classe est présentée en section 6.1.

4.4 Transition vers le Deep Learning

L'analyse des résultats ML révèle trois limites fondamentales :

1. Les features manuelles, conçues pour 4/5 classes, ont du mal à capturer les différences subtiles entre 13 types cellulaires (notamment MY vs. MMY vs. PMY, qui représentent différents stades de maturation d'une même lignée).

2. La segmentation imparfaite introduit souvent du bruit dans toutes les features en aval.
3. Le nombre de features à concevoir manuellement augmente avec le nombre de classes, et il devient difficile de trouver des descripteurs discriminants pour des classes rares comme PLY (11 images) ou PC (68 images).

Ces observations motivent notre passage au Deep Learning, où le réseau apprend ses propres représentations directement à partir des pixels, contournant les problèmes de segmentation et de choix de features.

5 Méthodologie : Pipeline Deep Learning

5.1 Approche initiale : MobileNetV2

On commence par utiliser l'architecture DL MobileNetV2, due à son faible nombre de paramètres (3.5M), permettant une exécution rapide. Le modèle pré-entraîné sur ImageNet est fine-tuné avec les paramètres suivants : images redimensionnées en 224×224 , batch size 64, learning rate 10^{-4} , AdamW optimizer avec *discriminative learning rates* (backbone à LR/10), CosineAnnealingWarmRestarts et early stopping (patience 7).

Le résultat initial est décevant : **F1 = 0.35** sur le leaderboard. On remarque que le modèle prédit quasi tout le temps les 2/3 classes majoritaires (SNE, LY), en ignorant les classes rares. On identifie deux problèmes : la capacité limitée de MobileNetV2 (3.5M paramètres, conçu pour l'inférence mobile) pour discriminer 13 classes morphologiquement proches, et une pondération des classes défectueuse dans la fonction de perte.

5.2 Passage à EfficientNet-B3 et correction des poids de classes

Face à ces deux problèmes, nous effectuons un double changement. D'une part, on remplace MobileNetV2 par **EfficientNet-B3** (12M paramètres, images 256×256), qui offre une capacité de représentation $3.5\times$ supérieure grâce à son architecture à *compound scaling* (profondeur, largeur et résolution optimisées conjointement). Ce choix est motivé par la nécessité d'un modèle plus expressif pour distinguer 13 classes, tout en restant raisonnable en coût computationnel comparé à un VGG16 (138M paramètres) ou un ResNet152.

D'autre part, l'investigation du deuxième problème révèle un bug critique dans le calcul des

poids de classes pour la fonction de perte. La normalisation utilisée ($w_i = N_{\text{total}}/N_i$, puis normalisée par la somme) produit des poids extrêmes :

Classe	Nb imgs	Poids brut	$\sqrt{\text{poids}}$
PLY	11	9.24	~ 3.5
PC	68	1.43	~ 1.4
LY	8 101	0.01	~ 0.3
SNE	13 015	0.01	~ 0.2
Ratio max/min		924 $\times$	~ 12 $\times$

FIGURE 3 – Poids de classes avant et après correction.

Avec les poids bruts, le modèle reçoit un signal 924 fois plus fort pour PLY (11 images) que pour LY (8 101 images), rendant l'apprentissage chaotique. On va ainsi utiliser la **racine carrée** des poids inverses ($w_i = \sqrt{N_{\text{total}}/N_i}$), réduisant le ratio à $\sim 12 : 1$, un équilibrage modéré qui ne néglige ni les classes rares ni les classes fréquentes.

On a également testé une *Focal Loss* (gamma = 2.0) et des techniques de *Mixup/CutMix*, mais ces approches amplifiaient le problème de déséquilibre ou brouillaient les frontières entre classes visuellement proches. La solution la plus efficace s'est avérée être la plus simple : une **CrossEntropy-Loss** standard avec label smoothing (0.1) et les poids corrigés en racine carrée.

Avec ce double changement (EfficientNet-B3 + poids corrigés), le F1 passe de 0.35 à **0.69** sur le leaderboard, un gain de **+0.34**, le plus important du projet.

5.3 Oversampling offline et 2-stage training (V4)

Pour aller au-delà de 0.69, on va cibler spécifiquement les classes rares qui tirent le F1 macro vers le bas. Deux techniques complémentaires sont mises en œuvre :

Oversampling offline : Chaque classe est dupliquée dans le DataFrame jusqu'à atteindre un minimum de 500 images. Combiné avec la data augmentation aléatoire (rotation 360°, jitter de couleur, crop), chaque copie est vue différemment à chaque époque, évitant le surapprentissage que causerait une simple répétition d'images identiques.

2-stage training : L'entraînement suit un protocole en deux stages :

1. **Stage 1** (35 époques) : entraînement sur le dataset oversamplé avec $\sqrt{\text{poids}}$, 3 époques de freeze du backbone puis fine-tuning complet. Le modèle apprend les features générales de toutes les classes.
2. **Stage 2** (15 époques, LR $\div 20$) : fine-tuning avec des poids renforcés pour les classes rares (puissance 0.75 au lieu de 0.5), et un sampler encore plus biaisé vers les classes minoritaires. Le learning rate très faible évite d'oublier ce qui a été appris au stage 1.

Avec cette stratégie, EfficientNet-B3 atteint **F1 = 0.7473** sur le leaderboard, une amélioration de +0.06 par rapport à la version sans oversampling.

5.4 Diversification des architectures

Pour exploiter la complémentarité entre architectures, on va entraîner trois modèles supplémentaires avec la même stratégie V4 :

- **ResNet50** (25M params) : architecture à skip connections, référencée par Habibzadeh et al. [5]. F1 = 0.65 seul.
- **ConvNeXt-Small** (50M params) : CNN moderne (2022) rivalisant avec les Vision Transformers. F1 = 0.7616.
- **Swin Transformer Tiny** (28M params) : Vision Transformer à attention spatiale par fenêtres glissantes. **F1 = 0.7675**, notre meilleur modèle individuel.

Chaque modèle est évalué avec *Test-Time Augmentation* (TTA) $\times 10$: 10 augmentations aléatoires sur les images test, dont les probabilités softmax sont moyennées pour une prédiction plus robuste.

5.5 Ensemble final

L'ensemble final combine les trois meilleurs modèles par **soft voting pondéré** : les probabilités softmax sont moyennées, chaque modèle étant pondéré par son score F1 individuel sur le leaderboard :

$$P_{\text{ensemble}}(y|x) = \frac{\sum_{k=1}^K w_k \cdot P_k(y|x)}{\sum_{k=1}^K w_k} \quad (1)$$

avec $w_{\text{Swin}} = 0.7675$, $w_{\text{ConvNeXt}} = 0.7616$, $w_{\text{EffNet}} = 0.7473$.

Ce dernier ensemble atteint **F1 = 0.7677**, une amélioration légère par rapport au Swin seul.

6 Expériences et Résultats

6.1 Résultats de la pipeline ML classique

La figure 4 présente les performances des six classifieurs évalués en validation croisée stratifiée (5-fold). La Figure 5 montre les matrices de confusion de deux classifieurs ML.

Classifieur	Accuracy	F1 macro
Random Forest	0.802	0.357
SVM (RBF)	0.789	0.459
LDA	0.781	0.393
K-PCA + SVM	0.690	0.297
Rég. Logistique	0.642	0.354
Decision Tree	0.625	0.281

FIGURE 4 – Performances des classifieurs ML (validation 5-fold).

L’observation la plus frappante est le décalage entre l’accuracy et F1 macro. Le Random Forest obtient la meilleure accuracy (0.802) mais seulement le quatrième F1 macro (0.357). En examinant sa matrice de confusion (Figure 5), on constate qu’il prédit correctement SNE (2 535/2 603) et LY (1 371/1 621), mais obtient un recall de 0% sur BNE, MMY, PMY et PLY. À l’inverse, le SVM (RBF) sacrifie un peu d’accuracy (0.789) pour mieux répartir ses prédictions, atteignant le meilleur F1 macro (0.459).

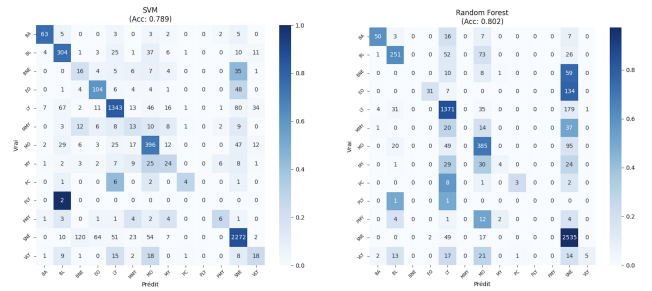
Analyse par classe. L’examen détaillé des matrices de confusion révèle trois catégories de classes :

- **Classes bien classifiées** ($F1 > 0.7$) : SNE et LY, les deux classes majoritaires, qui représentent 73% du dataset. Tous les modèles les classifient correctement.
- **Classes intermédiaires** ($F1 0.3\text{--}0.7$) : BA, BL, MO, EO, présentes en quantité suffisante (400–2 700 images) avec des caractéristiques visuelles distinctives.
- **Classes échouées** ($F1 < 0.2$) : BNE, MMY, MY, PC, PLY, PMY, VLY, trop rares et/ou visuellement trop proches d’autres classes. PLY (11 images) n’est jamais correctement prédit par aucun modèle.

Confusions fréquentes : Les matrices révèlent des confusions récurrentes et biologiquement cohérentes, en accord avec les difficultés identifiées par

Al-Dulaimi et al. [1] : BNE est confondu avec SNE (même lignée neutrophile, différenciés uniquement par la lobulation du noyau), MMY et MY sont confondus entre eux et avec MO (stades de maturation successifs de la lignée myéloïde), et VLY est confondu avec LY (variante morphologique du lymphocyte).

Score Kaggle : La soumission avec le SVM (meilleur F1 macro en validation) obtient un score de **0.4320** sur le leaderboard public, confirmant que les features manuelles, même riches (géométriques, LBP, LDP, PRiCoLBP, couleur, DT-CWT, bispectrales, L-moments), ne suffisent pas à discriminer 13 classes de WBC.



(a) SVM (RBF) — Acc : 0.789 (b) Random Forest — Acc : 0.802

FIGURE 5 – Matrices de confusion des deux meilleurs classifieurs ML (SVM et Random Forest)

6.2 Résultats de la pipeline Deep Learning

6.2.1 Progression itérative

La figure 6 résume la progression de nos scores sur le leaderboard public.

Étape	Méthode	F1
Pipeline ML	SVM	0.4320
DL V1	MobileNetV2	0.3500
DL V3	EfficientNet-B3	0.6900
DL V4	EffNet-B3 + over. + 2-stage	0.7473
DL	ResNet50 (V4)	0.6500
DL	ConvNeXt-Small (V4)	0.7616
DL	Swin-T (V4)	0.7675
Final	Ens. pond. top-3	0.7677

FIGURE 6 – Progression des scores F1 macro sur le leaderboard.

6.2.2 Analyse des gains

Chaque amélioration peut être attribuée à un changement important :

- **Correction des poids de classes** (V1 → V3, +0.34) : le gain le plus important du projet. Le passage de poids bruts à des poids en racine carrée permet au modèle d'apprendre toutes les classes.
- **Oversampling + 2-stage training** (V3 → V4, +0.06) : la duplication des classes rares à 500 images minimum, combinée à un second stage de fine-tuning avec des poids renforcés, améliore le F1 des classes minoritaires.
- **Architecture moderne** (EffNet → Swin, +0.02) : le Swin Transformer, grâce à son mécanisme d'attention spatiale, capture mieux les variations morphologiques subtiles entre classes proches.
- **Ensemble pondéré** (+0.002) : le gain marginal du soft voting suggère que les modèles font des erreurs similaires sur les classes les plus difficiles.

6.2.3 Comparaison ML vs DL

La confrontation directe entre les deux pipelines est éclairante :

- **Accuracy** : ML = 0.80 (RF) vs DL = 0.89 (Swin). Le DL gagne 9 points.
- **F1 macro** : ML = 0.46 (SVM) vs DL = 0.77 (Swin). Le DL gagne **31 points**.
- Le gain est bien plus important en F1 qu'en accuracy, car le DL améliore drastiquement les classes rares que le ML ignorait.

7 Discussion

7.1 Pourquoi le DL surpasse le ML classique

L'écart de performance entre ML classique (F1 = 0.43) et DL (F1 = 0.77) s'explique par trois facteurs : Les features manuelles, conçues pour 4/5 classes, capturent mal les différences subtiles entre 13 types cellulaires. La segmentation imparfaite introduit du bruit systématique. Le DL apprend des représentations directement depuis les pixels, s'adaptant à la complexité du problème. Ce résultat contraste avec Rustam et al. [2] qui montrent que le ML classique surpasse le DL sur 5 classes ; la différence réside dans le nombre de classes et le déséquilibre.

7.2 Classes problématiques

Les classes les plus difficiles à classer sont celles qui combinent rareté et similarité visuelle

avec d'autres classes :

- **PLY** (11 images) : trop rare pour qu'aucun modèle généralise, même avec oversampling.
- **PC** (68 images) et **PMY** (114 images) : représentent des stades de maturation intermédiaires, morphologiquement proches de BL et MY.
- **BNE** vs. **SNE** : les neutrophiles en bande et segmentés diffèrent principalement par le degré de lobulation du noyau, une distinction fine que les features manuelles capturent difficilement.

7.3 Le rôle critique de la pondération

La découverte la plus marquante de ce projet est l'impact dévastateur d'une mauvaise pondération des classes. Le passage de poids bruts à des poids en racine carrée a été le levier le plus décisif, plus que le choix de l'architecture ou de l'augmentation. Ce résultat rappelle que dans les problèmes fortement déséquilibrés, **la calibration de la loss est plus importante que la complexité du modèle**.

7.4 Limites

Plusieurs limites doivent être mentionnées :

- La contrainte d'utiliser uniquement des poids pré-entraînés sur ImageNet exclut des modèles spécifiquement entraînés sur des images médicales (histopathologie, cytologie), qui pourraient offrir de meilleures représentations de base.
- Le temps de calcul (GPU P100/3090 sur cluster SSH) a limité le nombre d'expériences possibles : un seul run prend 2-3 heures.
- L'oversampling par duplication, bien que bénéfique, risque de mémoriser les exemples rares plutôt que de généraliser ; des techniques de génération synthétique (SMOTE en espace latent, GANs) pourraient être plus efficaces.

8 Conclusion

Ce projet a abouti à un F1 macro de **0.7677** sur le challenge de classification de WBC en 13 classes, partant d'une baseline ML à 0.43. La progression résulte d'une démarche itérative : identification du problème des poids de classes (+0.34), oversampling offline des classes rares (+0.06), diversification des architectures (+0.02), et ensemble pondéré final (+0.002).

Les enseignements principaux sont :

- Dans les problèmes fortement déséquilibrés, la calibration de la loss est plus décisive que le choix du modèle.
- Les architectures modernes (Swin Transformer, ConvNeXt) apportent un gain systématique par rapport aux classiques (ResNet).
- L'ensemble de modèles n'améliore les résultats que si les modèles composants sont de qualité comparable.

Avec davantage de temps, on pouvait explorer plusieurs autres pistes : K-fold cross-validation pour l'entraînement DL, génération synthétique d'images pour les classes rares, modèles pré-entraînés sur des données médicales (si autorisé), et exploration de techniques de few-shot learning pour les classes à très peu d'exemples.

Références

- [1] K. Al-Dulaimi, J. Banks, V. Chandran, I. Tomeo-Reyes, and K. Nguyen, "Classification of White Blood Cell Types from Microscope Images : Techniques and Challenges," in *Microscopy Science : Last Approaches on Educational Programs and Applied Research*, 2019.
- [2] F. Rustam, N. Aslam, I. De La Torre Díez, Y. D. Khan, C. L. Rodríguez, et al., "White Blood Cell Classification Using Texture and RGB Features with Machine Learning," *Healthcare*, vol. 10, no. 11, p. 2230, 2022.
- [3] M. Ghosh, D. Das, S. Mandal, C. Chakraborty, M. Pal, A. K. Maity, S. K. Pal, and A. K. Ray, "Statistical Pattern Analysis of White Blood Cell Nuclei Morphometry," in *Proc. IEEE Students' Technology Symposium*, IIT Kharagpur, 2010.
- [4] S. H. Rezaatofghi and H. Soltanian-Zadeh, "Automatic Recognition of Five Types of White Blood Cells in Peripheral Blood," in *ICIAR*, 2010.
- [5] M. Habibzadeh, M. Jannesari, Z. Rezaei, H. Baharvand, and M. Totonchi, "Automatic White Blood Cell Classification Using Pre-trained Deep Learning Models : ResNet and Inception," in *Proc. ICMV*, Vienna, Austria, 2018.
- [6] S. Bikheth, A. M. Darwish, H. A. Tolba, and S. I. Shaheen, "Segmentation and Classification of White Blood Cells," in *Proc. IEEE Int. Conf. on Acoustics, Speech and Signal Processing*, 2000.